02. Feasify Comprehensive Software Test Plan

Document ID: FEASIFY-DOC-02

Target Application: Feasify Capstone Proposal & Feasibility System

Prepared By: Senior QA Lead & Test Orchestrator

Effective Date: November 11, 2026

Status: Approved for Formal Assessment

---

1. Introduction

This Software Test Plan defines the framework, scope, methodology, resources, and criteria utilized to evaluate the **Feasify** academic capstone management application. It establishes an evidence-based quality verification process to determine readiness for academic defense and production staging.

---

2. Project Overview

Feasify streamlines the capstone proposal and feasibility evaluation process for academic business and information technology departments. It manages proposal drafting, registration compliance checks, financial statement balancing, and AI-assisted rubric evaluations across Student, Adviser, and Chairperson roles.

---

3. Testing Objectives

1. **Verify Functional Correctness:** Confirm that all implemented features operate according to defined academic and business specifications.
2. **Validate Business Constraints:** Strictly test the maximum 3-proposal limit per student team across 10 distinct edge-case scenarios.
3. **Audit Security & Authorization:** Assess role-based access control (RBAC), token verification, IDOR risks, and Firestore client-side boundaries.
4. **Stress & Concurrency Evaluation:** Quantify system stability, throughput, and response latency under graduated concurrency levels (1 to 100 virtual users).
5. **Evaluate Financial Calculations:** Validate balance sheet equality ($Assets = Liabilities + Equity$), BMBE vs. CREATE tax calculations, and break-even revenue formulas.
6. **Ensure Cross-Module Integrity:** Verify real-time status transitions and notifications across student submissions and adviser evaluations.

---

4. Scope of Testing

In-Scope

- Frontend UI components, navigation, and modal forms (`Feasify/src/`).

- Express 5 backend REST APIs, Socket.IO handlers, and middleware (`feasify-backend/`).
- Firebase Authentication and role segregation (`ProtectedRoute.tsx`).
- Cloud Firestore collections: `users` , `groups` , `proposals` , `notifications` , `audit_logs` , `messages` .
- DTI BNRS and SEC eSPARC name checking logic and trademark alerts (`services/nameChecker/`).
- Leaflet and Nominatim Philippine location picking (`LocationPickerMap.tsx`).
- 5-year financial statement projections, break-even math, and BMBE tax logic.
- AI feasibility analysis prompts, cascade failover, and rubric scoring.
- Automated tests, TypeScript compiler verification, and ESLint configuration.

Out-of-Scope

- Live payment gateway processing (none exists in Feasify).
- Third-party live DTI/SEC enterprise API production traffic (requires state enterprise accreditation).
- Third-party production Firebase project destruction or table deletions.

---

5. Testing Approach

A hybrid multi-agent testing strategy is employed combining:

- **Static Code Analysis:** AST inspections, TypeScript compilation checks, and configuration audits.
- **Automated Integration & Unit Testing:** Fast, reproducible Node.js test runners executing assertion matrices.
- **Stress & Load Testing:** Concurrency test harnesses measuring requests per second, p95 latency, and error rates.
- **Dynamic Security Analysis:** Non-destructive unauthorized payload delivery, IDOR probing, and rate limiter fuzzing.
- **Cross-Layer Data Tracing:** Correlating frontend triggers with Firestore mutations and backend API responses.

---

6. Testing Environment

Parameter	Configuration
Operating System	Microsoft Windows 11 (OS Architecture: x64)
Node.js Version	v22.14.0
Frontend Test Server	Vite Dev Server (port 5173) / Vite Production Preview
Backend Test Server	Express 5 HTTP Server (ephemeral test port / port 10000)
Test Database Mode	Firebase Emulator / Development Sandbox & In-Memory Replicas
Sandbox Flags	`DTI_SANDBOX_MODE=true` , `SEC_SANDBOX_MODE=true`

---

7. Testing Tools & Frameworks

- **Node.js Test Engine:** Node.js built-in `node:test` and `node:assert` modules.
- **HTTP Client:** Node.js `http` client with keep-alive and chunked parsing.
- **Performance Profiling:** `perf_hooks.performance` high-resolution timer.
- **Type Checking:** TypeScript Compiler `tsc` (v5.9.3) via `tsconfig.app.json`.
- **Linting:** ESLint 9 Flat Config runner.
- **Frontend Bundler:** Vite 7.3.2 rollup build verification.

---

8. Roles and Responsibilities (Multi-Agent Team)

Agent ID	Specialized Role	Assigned Responsibility
Agent 01	Architecture Auditor	Inspect tech stack, component mapping, and dependency graph
Agent 02	Frontend Functional Tester	Verify buttons, inputs, modals, toasts, and UI views
Agent 03	Backend & API Tester	Audit endpoints, HTTP status codes, and JSON response schemas
Agent 04	Database Auditor	Audit Firestore schemas, queries, indexes, and write paths
Agent 05	Security Specialist	Test RBAC boundaries, token verification, and IDOR vulnerabilities
Agent 06	Proposal Specialist	Verify 3-proposal limit, draft saving, and submission workflows
Agent 07	Name Checker Specialist	Test DTI/SEC providers, trademark rules, and search fallback
Agent 08	UI/UX Specialist	Inspect design system, black-text rules, and responsiveness
Agent 09	Performance Engineer	Benchmark concurrency (1 to 100 VUs), measure latency & req/s
Agent 10	Integration Tester	Validate end-to-end student, adviser, and chairperson data flow
Agent 11	Automation Engineer	Scaffold, execute, and verify automated backend test suites
Agent 12	Documentation Specialist	Consolidate verified evidence into formal QA documents

---

9. Entry and Exit Criteria

Entry Criteria

- Codebase builds successfully with no catastrophic bundling failure.
- Backend dependencies installed and runnable in sandbox mode.
- Access to all project source files across frontend and backend.

Exit Criteria

- 100% of planned automated test suites executed.
- All verified defects categorized with reproducible steps and root causes.

- Stress testing completed across all 5 planned concurrency levels.
- Complete documentation package generated in `/docs/testing/`.
- No modification of existing production business logic during audit phase.

---

10. Risks, Assumptions, and Mitigations

Risk	Impact	Mitigation Strategy
External DTI/SEC Outage	External service cannot be queried live	Use verified sandbox mocking with real government portal URL generation
Production Data Corruption	Risk of overwriting live capstone proposals	Use isolated ephemeral test identifiers and non-destructive assertions
API Rate Quota Depletion	Gemini API calls blocked by rate limiter	Test multi-model cascade with fallback and exponential backoff retry
Firestore Read Cost Spikes	Unbounded listeners fetch entire collections	Identify N+1 and unbounded queries in audit; execute tests against localized mocks